Probing the Latent Space of Video Diffusion U-Nets

Disentangling Content vs. Motion

Team Claudn't · revised 22 September 2026 · supersedes the draft of 21 September

1. Project snapshot

Team: Claudn't · **Roll numbers:** [fill in, one per member]

The task. A video diffusion U-Net does two jobs in alternation. Its spatial layers work inside a single frame and decide what the thing looks like; its temporal layers work across frames and decide how it moves. Our job is to find out where that division actually sits inside the network, which layers and which denoising timesteps, and then prove we found it by making an edit that only that understanding would allow. The target demonstration is a clip of a golden retriever running on a beach where we keep the exact running gait but turn the dog into a horse, and the reverse, where the dog stays but starts walking instead.

Why this is a probing project and not a generation project. We are not trying to build a better video model. We are trying to explain one. The edit at the end is evidence, not the product. That framing matters for how we spend compute: a careful sweep over a small, controlled clip set beats a large, messy one.

What changed in this revision. The previous draft left roughly twenty methodological choices implicit — the distance function behind the central figure, how pairs are drawn, how many seeds, what counts as success, and what the "trained model" deliverable actually is. Every one of those is now fixed in writing. Section 5.7 is the register; it is the section to read if you are reviewing this document against the earlier one.

Timeline. Course staff extended the overall project by roughly a month and a half, which puts the intensive assessment somewhere in early-to-mid November 2026. We have not seen a firm date yet, so everything past 1 November in the table in section 6 is provisional. The prep presentation deadline (21 September) may or may not have moved with it; we are treating it as still live and submitting on time.

2. Where we are: 2 to 22 September

We started on 2 September. The window from 11 to 20 September ran into midsems, so we deliberately planned almost nothing for it apart from reading, and that is how it went. Everything below is what actually happened in those three weeks.

2.1 Reading completed

Groups 1 to 5 of the reading list in section 3 are done. Group 7 (the model and tooling documentation) we are reading as we build, which is the sensible order for it. The single most useful thing we read was not on the original list at all: a small cluster of recent papers that have already gone looking for motion inside video diffusion features. They changed what we plan to measure. See section 5.2.

2.2 Pilot experiments

Five short experiments on a single GPU, mostly to find out whether the project is tractable at all before committing to a method. E1 to E4 are complete. E5 is new in this revision and is scheduled for Stage 1, because it is the feasibility check for the trained-model deliverable and we would rather fail it in September than in late October.

ID	QUESTION WE WERE ASKING	WHAT WE RAN	OUTCOME
E1	**Can we generate video at all on the hardware we have?**	AnimateDiff v1.5 (motion adapter `v1-5-2`) on an SD1.5 backbone through `diffusers`. 16 frames, 512×512, 25 DDIM steps, fp16, CFG 7.5.	**Works. 94 s per clip, 10.2 GB peak VRAM on a Kaggle T4.** This is the unit cost behind every budget in section 7. Our 8 GB laptop GPU cannot run the configuration, so all generation happens on Kaggle.
E2	**If we fix the seed and change one word of the prompt, does the rest of the clip stay put?**	Same latent noise, same settings, prompt varied along one axis at a time: noun swap (retriever → cat) and verb swap (running → walking).	Largely yes. Composition and camera framing survive a fixed seed well enough that paired clips are a usable controlled design. **The failure case is informative and we have designed around it:** a noun swap between very differently shaped subjects drags the layout with it. See D-05.
E3	**Can we get activations out, and how much disk does that cost?**	Forward hooks on `unet.up_blocks[i].motion_modules[j]` and on the `attn2` cross-attention modules. Dumped shapes and sizes for one clip.	Straightforward to hook. **Storage is the real constraint, not compute:** a full fp16 dump of every module at every timestep is 3.4 GB per clip, which is 1.2 TB across the planned set. The harness therefore stores summaries by default and full tensors only on request (D-09).
E4	**Do the temporal modules causally carry motion, or is that just an architectural story?**	Scaled the motion-module residual contribution down towards zero and regenerated with an otherwise identical configuration.	The clip collapses towards a near-static sequence while each individual frame stays a plausible image. That is the cheapest possible causal demonstration that these modules are the cross-frame path, and it is a good opening result for the report.
E5 NEW	**Can we train a LoRA on this hardware, and at what clip length and resolution?**	*Stage 1, 21–27 September.* One motion LoRA, one concept, 100 steps, rank 16, gradient checkpointing on. Purpose is to record frames × resolution that fit in 16 GB, seconds per step, and peak VRAM — nothing else.	*Pending.* E1 established the inference budget before we committed to a sweep; E5 does the same for the training budget before we commit to section 5.5. The expected answer is that training runs at 8 frames × 256×256 even though we generate at 16 × 512.

2.3 What we decided because of the pilots

- **AnimateDiff is the primary model.** E1 settled the feasibility question and E4 confirmed the motion module is a real causal handle, not just a label on a box.
- **Every clip in the probing set is generated from a fixed seed and a fixed noise tensor.** E2 is the justification. Without it, activation differences between two clips are dominated by sampling noise and the whole probe is meaningless. How many seeds, and how they are assigned, is now fixed in D-04.
- **The harness stores per-coordinate summary statistics by default.** E3 killed the original plan of dumping every activation to disk, at 3.4 GB per clip.
- **Content pairs are drawn within a body-plan family.** E2's noun-swap failure is a direct threat to the controlled design, so the subject vocabulary is split into two families and cross-family pairs are reported separately as a labelled hard subset. See D-05.

- **The trained-model deliverable gets a feasibility pilot of its own.** E5, above.

3. Reading roadmap

Grouped by subtopic, roughly in the order we read them. Citations follow the course format.

Group 1. Video diffusion foundations

Ho, Jonathan, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi and David J. Fleet. **Video Diffusion Models**, NeurIPS, 2022. https://arxiv.org/abs/2204.03458

Blattmann, Andreas, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler and Karsten Kreis. **Align your Latents: High-Resolution Video Synthesis with Latent Diffusion Models**, CVPR, 2023. https://arxiv.org/abs/2304.08818

Blattmann, Andreas, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani and Robin Rombach. **Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets**, arXiv, 2023. https://arxiv.org/abs/2311.15127

> These establish the factorisation we are probing: spatial blocks inside a frame, temporal blocks across frames, repeated at every resolution.

Group 2. Motion modules and separation by design

Guo, Yuwei, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin and Bo Dai. **AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models without Specific Tuning**, ICLR, 2024. https://arxiv.org/abs/2307.04725

Zhao, Rui, Yuchao Gu, Jay Zhangjie Wu, David Junhao Zhang, Jiawei Liu, Weijia Wu, Jussi Keppo and Mike Zheng Shou. **MotionDirector: Motion Customization of Text-to-Video Diffusion Models**, ECCV, 2024. https://arxiv.org/abs/2310.08465

Wu, Jay Zhangjie, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie and Mike Zheng Shou. **Tune-A-Video: One-Shot Tuning of Image Diffusion Models for Text-to-Video Generation**, ICCV, 2023. https://arxiv.org/abs/2212.11565

> Closest prior art to our deliverable. MotionDirector's dual-path appearance/motion LoRA is both the baseline we compare against and, in modified form, the recipe behind our own trained model in section 5.5. It shows the split is exploitable, which is a weaker claim than ours but the same territory.

Group 3. Activation patching as a method

Meng, Kevin, David Bau, Alex Andonian and Yonatan Belinkov. **Locating and Editing Factual Associations in GPT**, NeurIPS, 2022. https://arxiv.org/abs/2202.05262

Wang, Kevin, Alexandre Variengien, Arthur Conmy, Buck Shlegeris and Jacob Steinhardt. **Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 Small**, ICLR, 2023. https://arxiv.org/abs/2211.00593

Zhang, Fred and Neel Nanda. **Towards Best Practices of Activation Patching in Language Models: Metrics and Methods**, ICLR, 2024. https://arxiv.org/abs/2309.16042

> Developed on language models, but the causal logic carries over unchanged. Zhang and Nanda drive two decisions in this document: our corruption is resampling rather than noising (D-12), and the distance metric behind the central figure is written down before the sweep rather than chosen after seeing the data (D-01).

Group 4. Cross-attention as a diagnostic

Hertz, Amir, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch and Daniel Cohen-Or. **Prompt-to-Prompt Image Editing with Cross Attention Control**, ICLR, 2023. https://arxiv.org/abs/2208.01626

Tang, Raphael, Akshat Pandey, Zhiying Jiang, Gefei Yang, Karun Kumar, Jimmy Lin and Ferhan Ture. **What the DAAM: Interpreting Stable Diffusion Using Cross Attention**, ACL, 2023. https://arxiv.org/abs/2210.04885

> Working recipes for pulling attention maps out of a diffusion U-Net per layer and per timestep. We adapt DAAM's aggregation from one image to a stack of frames by keeping the frame axis instead of collapsing it.

Group 5. Where motion has already been located (added after the pilots)

Xiao, Zeqi, Yifan Zhou, Shuai Yang and Xingang Pan. **Video Diffusion Models are Training-free Motion Interpreter and Controller**, NeurIPS, 2024. https://arxiv.org/abs/2405.14864

Yatim, Danah, Rafail Fridman, Omer Bar-Tal, Yoni Kasten and Tali Dekel. **Space-Time Diffusion Features for Zero-Shot Text-Driven Motion Transfer**, CVPR, 2024. https://arxiv.org/abs/2311.17009

Ling, Pengyang, Jiazi Bu, Pan Zhang, Xiaoyi Dong, Yuhang Zang, Tong Wu, Huaian Chen, Jiaqi Wang and Yi Jin. **MotionClone: Training-Free Motion Cloning for Controllable Video Generation**, ICLR, 2025. https://arxiv.org/abs/2406.05338

Baherwani, Vatsal, Yixuan Ren and Abhinav Shrivastava. **Characterizing Motion Encoding in Video Diffusion Timesteps**, arXiv, 2025. https://arxiv.org/abs/2512.22175

Kwon, Mingi, Jaeseok Jeong and Youngjung Uh. **Diffusion Models already have a Semantic Latent Space**, ICLR, 2023. https://arxiv.org/abs/2210.10960

> This group is the reason our plan changed. Together these papers tell us roughly where to look before we start looking, which turns a blind sweep into a set of testable predictions. Details in section 5.2.

Group 6. Earlier framings of the same split

Tulyakov, Sergey, Ming-Yu Liu, Xiaodong Yang and Jan Kautz. **MoCoGAN: Decomposing Motion and Content for Video Generation**, CVPR, 2018. https://arxiv.org/abs/1707.04993

Yu, Sihyun, Weili Nie, De-An Huang, Boyi Li, Jinwoo Shin and Anima Anandkumar. **Efficient Video Diffusion Models via Content-Frame Motion-Latent Decomposition**, ICLR, 2024. https://arxiv.org/abs/2403.14148

> MoCoGAN is where the content/motion vocabulary in our project brief comes from, and CMD is the modern diffusion-era version of the same idea. Both build the split in by construction. We are asking the harder question: does a model that was never factored this way learn a version of it anyway?

Group 7. Tools and documentation (read while building)

Teed, Zachary and Jia Deng. **RAFT: Recurrent All-Pairs Field Transforms for Optical Flow**, ECCV, 2020. https://arxiv.org/abs/2003.12039

Karaev, Nikita, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi and Christian Rupprecht. **CoTracker: It is Better to Track Together**, ECCV, 2024. https://arxiv.org/abs/2307.07635

Huang, Ziqi, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao and Ziwei Liu. **VBench: Comprehensive Benchmark Suite for Video Generative Models**, CVPR, 2024. https://arxiv.org/abs/2311.17982

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang and Weizhu Chen. **LoRA: Low-Rank Adaptation of Large Language Models**, ICLR, 2022. https://arxiv.org/abs/2106.09685

Soomro, Khurram, Amir Roshan Zamir and Mubarak Shah. **UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild**, arXiv, 2012. https://arxiv.org/abs/1212.0402

Hugging Face. **Diffusers documentation, text-to-video pipelines.** https://huggingface.co/docs/diffusers/using-diffusers/text-img2vid. Accessed on: 21 September 2026.

Hugging Face. **PEFT documentation, LoRA methods.** https://huggingface.co/docs/peft/developer_guides/lora. Accessed on: 22 September 2026.

Guo, Yuwei et al. **AnimateDiff model card.** https://huggingface.co/guoyww/animatediff. Accessed on: 21 September 2026.

4. Model and dataset shortlist

Primary model: AnimateDiff. An SD1.5 UNet (roughly 860M parameters, frozen) with a separately trained motion module inserted after its residual and attention blocks. The motion module checkpoint `mm_sd_v15_v2` is about 417M parameters, around 1.6 GB on disk. The earlier draft of this document said the motion module was 1.7B; that was wrong, it confused the file size in gigabytes with a parameter count. The total is closer to 1.3B parameters before the VAE and text encoder.

We picked it because the spatial/temporal division is already explicit in the code, which means Deliverable 2 starts from a hypothesis rather than a blank sheet. The honest caveat, and it belongs in the report: because the motion

module was trained on top of a frozen image model, the "content" layers never saw motion during training. Any separation we find is partly an artefact of that training procedure. This is exactly why the ModelScope check matters.

Exact revisions. Pinned at the start of Stage 2 and recorded in the run manifest, because `runwayml/stable-diffusion-v1-5` was withdrawn from the Hub and the pipeline now resolves through a mirror. The resolved base repository id and commit hash are written into every result file rather than assumed (D-20).

MODEL	SIZE	WHY IT IS ON THE LIST	WHY IT IS NOT PRIMARY
AnimateDiff PRIMARY	417M motion module + ~860M SD1.5 UNet	Explicit spatial/temporal split in the module tree; 10.2 GB measured on a T4; mature `diffusers` support	Motion module bolted onto a frozen image model, so the split is partly given rather than learned jointly
ModelScope T2V `damo-vilab/text-to-video-ms-1.7b`	1.7B	Spatial and temporal trained together end to end. The harder and more interesting test case	Thinner tooling for attention extraction than SD-derived pipelines
Stable Video Diffusion (img2vid)	~1.5B UNet	Best output quality of the three, well documented	Image-conditioned only, so we cannot build matched content/motion pairs from text prompts

Plan: all of Deliverables 1 to 5 on AnimateDiff. If Stage 7 has room, repeat the Deliverable 2 sweep only on ModelScope T2V as a robustness check. Deliverable 4 asks us to demonstrate disentanglement, not to demonstrate it twice, so this is a bonus rather than a requirement.

4.1 The probing set, fully specified

We generate our own clips. That is not a shortcut; it is the only way to get pairs that differ along exactly one axis, and without that the probe has nothing to measure. The previous draft gave the shape of this set but not its contents. Both are now fixed, because a sweep run against an unwritten vocabulary is not reproducible.

The frozen vocabulary

15 subjects × 4 verbs × 2 scenes = **120 clips per seed**. The prompt template is `"a {subject} {verb} {scene}, high quality, detailed"` with the negative prompt `"bad quality, worse quality, blurry"` held constant. Subjects are split into two body-plan families so that content pairs can be drawn within a family (D-05).

AXIS	VALUES, FROZEN 22 SEPTEMBER 2026
Subject — family Q quadrupeds, 8	golden retriever · german shepherd · horse · wolf · tiger · fox · deer · brown bear
Subject — family H humans, 7	young man · young woman · athlete · soldier · dancer · child · elderly man
Verb (4)	running · walking · jumping · spinning
Scene (2)	on a beach · in a forest
Anchors	subject `golden retriever`, verb `running` — every pair holds one of these fixed

All four verbs are plausible for all fifteen subjects, which is why this particular list was chosen: a verb that only applies to some subjects would silently turn a motion pair into a content pair.

How a pair is built

A pair is two clips that share a seed, a noise tensor, a sampler and every prompt token but one. The anchor design makes the count deterministic rather than "whichever pairs we happened to use".

- **Content pairs.** Hold verb and scene, swap the subject against the family anchor. Within family Q: 7 partners × 4 verbs × 2 scenes = 56. Within family H the anchor is `young man`: 6 × 4 × 2 = 48. **Total 104 within-family content pairs.**
- **Motion pairs.** Hold subject and scene, swap the verb against the anchor `running`: 3 partners × 15 subjects × 2 scenes = **90 motion pairs.**
- **Cross-family content pairs.** A separate labelled set of 56 pairs (quadruped against human, same verb and scene). These are the hard subset E2 warned us about, reported separately and never pooled into the headline figure.

Seeds

The whole grid is regenerated under **three seeds — 42, 1337, 2024** — giving 360 clips. A pair always lives inside one seed. The selectivity score is computed within a seed and then averaged across seeds, and the spread across seeds is the error bar on the central figure. One seed would leave us unable to say whether the map is a property of the model or of one noise tensor, which is the first question any examiner will ask. At 94 s per clip this costs 9.4 GPU-hours, which section 7 shows we can afford.

Where the vocabulary comes from

- **Subjects and scenes** sampled from MSR-VTT and WebVid-10M captions, so we are not testing on a handful of toy prompts. WebVid in particular is close to what these models were pretrained on.
- **Verbs** taken from UCF101 and Kinetics-400 action labels, which give a clean standard list of distinct motions instead of whatever we happened to think of.

> **Data provenance, stated precisely.** For the probing set of Deliverables 1 to 4 we train on none of these datasets; they are a source of vocabulary and structure only, and every clip we probe is generated by the model itself. **Section 5.5 is the one exception** and we flag it rather than let it be discovered: the LoRA pair is fitted on roughly 20 real clips drawn from UCF101 and Kinetics-400. That is a deliberate, bounded use of real video for the trained-model deliverable, it is disclosed in the report's data section, and it touches nothing in Deliverables 1 to 4.

Pretraining data of the models we use

Required by the presentation checklist and worth stating plainly. The SD1.5 backbone was trained on LAION-5B subsets (image–text pairs, web-scraped, known to carry demographic and aesthetic bias). The AnimateDiff motion module was trained on WebVid-10M (short stock-footage clips, watermarked, biased towards professionally shot content and slow camera motion). Both biases are visible in our own pilots — the beach scenes look like stock footage — and they bound what our conclusions can claim: we are characterising *this* model, not video diffusion in general.

5. Refined plan, deliverable by deliverable

This section is where the reading in group 5 changed things. The original plan was to sweep every layer and every timestep and look for structure. That is still the shape of it, but we now know enough to predict what we should find, which makes the result falsifiable instead of merely descriptive. There are now **five** deliverables: the trained model in 5.5 has been promoted from an afterthought to a numbered workstream, because it is a graded item and because, done the way described there, it tests Deliverable 2 rather than merely sitting beside it.

5.1 Deliverable 1: setup and the extraction harness

- **Hook targets.** In `diffusers`, AnimateDiff builds a `UNetMotionModel` from a `UNet2DConditionModel` plus a `MotionAdapter`. The temporal path is the `motion_modules` list inside each `down_blocks[i]`, `up_blocks[i]` and the mid block — 21 modules in total (8 down, 1 mid, 12 up). The spatial path is the `transformer_blocks`, with `attn1` self-attention and `attn2` cross-attention to the text embedding. Forward hooks on both, keyed by module path.
- **Coordinate system.** Everything is indexed by a triple: (module path, denoising timestep, clip id). Every later stage queries by that triple, so getting it right once saves a lot of pain.
- **The two-tier sweep.** The coarse pass takes one representative module per block — 9 temporal coordinates and 7 spatial — at all 25 timesteps. The refined pass expands only inside whichever block wins, at full module resolution. This is what makes the sweep affordable; see D-07 and D-08.
- **What gets stored.** Not the raw tensors, by default: E3 measured a full dump at 3.4 GB per clip. Instead we store, per coordinate, the channel means and variances, the norms of the frame-mean-removed residual, and cross-attention maps head-averaged and downsampled to a 16 × 16 spatial grid with the frame axis kept, for the subject, verb and scene tokens only. Full tensors are retained for a short list of coordinates we mark as interesting (D-09).
- **Determinism.** One seed policy, written down and enforced in the generation script rather than passed by hand. Every clip records the seed, sampler, step count, resolution, base-model repository id and commit hash alongside its activations.
- **Attention maps.** DAAM-style aggregation, extended to video by keeping the frame axis rather than collapsing it. A token's map becomes a small movie, which is what makes it useful for telling content attention apart from motion attention.

5.2 Deliverable 2: three predictions, then a measurement

Rather than sweeping blind, we go in with three hypotheses drawn from the group 5 papers. Each is falsifiable and each is cheap to test with the harness from Deliverable 1.

H1, the layer axis. Motion information concentrates in the temporal modules at medium resolution rather than at the extremes. Xiao et al. find their motion feature works best after the first up-block and report that mid-resolution features give better motion correspondence than either shallow or deep ones. *Prediction:* motion selectivity peaks around `up_blocks[1]`, not at the highest-resolution decoder layers and not at the bottleneck.

H2, the timestep axis. Motion is decided early, appearance late. This is a widely repeated piece of folklore that Baherwani et al. actually went and tested, finding two regimes with a reasonably consistent boundary between them. *Prediction:* motion selectivity is high in the first third of the denoising trajectory and falls off; content selectivity does the opposite. If the boundary sits somewhere different for AnimateDiff than they report, that is a result worth writing up, not a bug.

H3, the content confound. Raw activations, even in temporal layers, are dominated by appearance. Xiao et al. only get a clean motion signal after subtracting the per-frame mean across the frame axis, which removes the static component. *Prediction:* if we skip that normalisation, every layer will look content-selective and we will wrongly conclude there is no motion anywhere. This one matters most, because it is the mistake we would otherwise have made in week one of the sweep.

How H1 and H2 are actually decided

A prediction with no test statistic is not falsifiable in practice. For H1, the test is the `argmax` over blocks of the mean motion selectivity, with a 95% bootstrap confidence interval over pairs; H1 survives if that interval excludes both the shallowest and the deepest block. For H2, we fit a single changepoint over the timestep axis and report its location with a bootstrap interval; H2 survives if the changepoint falls in the first half of the trajectory. Both tests are written into the analysis script before the sweep runs, not chosen after seeing the map.

The measurement, and the distance function behind it

The previous draft defined the central figure in terms of "how far the activations move" without saying what distance that is. Since the whole project reduces to one heat map computed from that function, it is now specified. Let *a* be the cached activation at coordinate (*L*, *t*) for one clip, with a frame axis of length *F* = 16. Split it into a static and a dynamic part:

$$\bar{a} = \text{mean}_F(a) \quad \cdot \quad \tilde{a} = a - \bar{a}$$

the static component, broadcast over frames, and the frame-mean-removed residual

For a pair of clips (*A*, *B*) the distance is one minus cosine similarity on the flattened tensor, taken on **ā for the content channel and ã for the motion channel**. Cosine rather than L2, because activation magnitudes differ by orders of magnitude between blocks and we need coordinates to be comparable; L2 is reported in an appendix as a robustness check, per Zhang and Nanda's advice that the metric should be fixed in advance and its alternatives shown.

This yields **two maps, not one**, which is the concrete form H3 takes:

$$S_{dyn}(L, t) = (d_c^{\tilde{a}} - d_m^{\tilde{a}}) / (d_c^{\tilde{a}} + d_m^{\tilde{a}}) \qquad S_{stat}(L, t) = (d_c^{\bar{a}} - d_m^{\bar{a}}) / (d_c^{\bar{a}} + d_m^{\bar{a}})$$

d_c is the distance across a content pair, d_m across a motion pair; both maps land in [−1, 1]

The sign is what makes these useful. Positive means the coordinate responds to what is depicted, negative means it responds to how things move, near zero means it responds to both or to neither. **S_{dyn} is the headline figure** — motion isolated from the static component — and S_{stat} is printed beside it as the control that shows what H3 predicted: without the split, almost everything reads as content-selective. Publishing both is the difference between demonstrating the confound and merely asserting it.

Aggregation. The ratio is computed per pair and then averaged over pairs, never the other way round: a ratio of averages is not an average of ratios, and only the per-pair form supports a bootstrap interval. We report the median over pairs with a 95% bootstrap CI, within each seed, then average across the three seeds.

The correlational counterpart

Separately, fit a small linear probe on each coordinate's cached activation to predict the motion verb and, in a second probe, the subject family. Probe accuracy per coordinate says what information is *present*. The patching in Deliverable 3 says what information is *used*. These two can disagree, and where they disagree is the most interesting thing we are likely to find.

Protocol, because a probe on 120 clips overfits trivially. Logistic regression with L2 regularisation on the flattened, frame-mean-removed activation, reduced to 256 dimensions by PCA fitted on the training split only. The split is **by scene**: train on `on a beach`, test on `in a forest`, then swap and average. This forces the probe to find something about the verb rather than memorising a scene. The verb probe is 4-way (chance 25%) with 60 test clips per fold; the subject-family probe is binary (chance 50%). Chance level is plotted on every probe figure.

5.3 Deliverable 3: the patching framework

A patch replaces the activation at a chosen coordinate during clip B's generation with the one cached from clip A. Both runs share an initial noise tensor or the comparison is meaningless. Following Zhang and Nanda, our corruption is **resampling** — the replacement comes from a real second clip — and not Gaussian noising. That choice is stated because the two answer different questions and are routinely confused.

Four granularities, in increasing order of precision. We implement them in this order and stop when one works well enough:

- **1. Whole tensor.** Replace the full activation. Blunt, and likely to break coherence, but it establishes an upper bound on effect size.
- **2. Channel subset.** Replace only the most motion-selective channels. This needs a per-channel score, which S(L, t) does not provide because it aggregates over channels, so we define $S_c(L, t, c)$ by the same formula restricted to channel *c* of the dynamic component and take the top *k*% by that score. *k* is swept over {1, 5, 10, 25} and reported as a curve (D-10). We expect this is where the method starts working.
- **3. Attention weights only.** Replace the temporal attention map rather than the features, in the spirit of MotionClone's observation that the dominant components of temporal attention are what actually drive motion.
- **4. Scaled residual injection.** Blend rather than replace, with a strength α from 0 to 1. Sweeping α over {0, 0.25, 0.5, 0.75, 1.0} gives a dose–response curve, which is far stronger evidence than a binary worked/did-not-work.

The patch window. A patch is applied over a contiguous range of timesteps, not a single one, and the range is a free parameter we sweep rather than a detail we leave unsaid: {one step}, {five steps centred on the peak}, {the first third}, {all 25}. Layer granularity is one module or one whole block. The winning (window, block) combination from the coarse pass is what Deliverable 4 uses.

Three controls, all of which we run before believing any positive result:

- **Null patch.** Patch a clip with its own activation; the output must come back unchanged. In fp16 with cuDNN autotuning it will not be bit-identical, so the criterion is numerical rather than exact: mean absolute difference of the final latents below 1e-3 *and* PSNR above 45 dB between the decoded clips. Below that bar there is a bug in the harness and nothing downstream is trustworthy (D-11).
- **Random coordinate.** Patch a coordinate with low |S|. The effect should be small. If a random layer does as well as our chosen one, we have not localised anything.
- **Symmetry.** Patch A into B and B into A. The two effects should be approximate inverses. Asymmetry means something is leaking.

5.4 Deliverable 4: the demonstration, and how we score it

Two edits per test case, each with a clear target: keep A's motion while taking B's content, and the reverse. **Twelve test pairs, both directions, so 24 edits** — six from family Q, six from family H, spanning all four verbs, listed in the appendix of the report so the selection cannot be read as cherry-picked. Reported as a video grid plus numbers.

Content fidelity s_c. CLIP similarity between output frames and the intended content prompt, plus CLIP or DINO image-to-image similarity against the reference clip's frames. The image-to-image version is the more honest of the two, since CLIP text similarity is easy to satisfy accidentally, and it is the one that enters the headline number.

Motion fidelity s_m. A trajectory-based score in the style of Yatim et al.: extract point tracks from both clips with CoTracker and compare displacement vectors over time. Optical flow correlation via RAFT as a cheaper second opinion.

Video quality. Three VBench dimensions — subject consistency, motion smoothness, temporal flickering — so we can show the edit did not simply wreck the clip. Cheap to run, and it pre-empts the obvious objection.

The headline number

A single disentanglement score built from the four cross-similarities of one edit. For an output that should carry A's motion and B's content:

$$D = \tfrac{1}{2}[\, s_c(\text{out}, B) + s_m(\text{out}, A) \,] - \tfrac{1}{2}[\, s_c(\text{out}, A) + s_m(\text{out}, B) \,]$$

s_c is content similarity, s_m is motion similarity, out is the edited clip

A correction the previous draft needed. As written, D adds a CLIP cosine (empirically ≈ 0.5 to 0.95) to a trajectory correlation (−1 to 1). Those are not commensurable and their sum is dominated by whichever has the wider spread. Both terms are therefore **min–max normalised across the whole evaluation set** before entering D, so D lands in [−1, 1] and the two axes contribute equally by construction. The raw un-normalised s_c and s_m are printed in the same table, so the normalisation hides nothing (D-14).

Positive and large means the edit moved the axis we aimed at and left the other one alone. Reported separately for both edit directions, because a method that works one way and not the other has not demonstrated disentanglement.

What would count as success, decided in advance

The previous draft never said what result would make us claim the project worked, which leaves the claim free to be fitted to whatever came out. Pre-registered, the claim is supported if **all three** hold:

- median D over the twelve pairs is greater for patching than for the prompt-only baseline, and the 95% bootstrap CI of that difference excludes zero;
- this holds in **both** edit directions, not just one;
- VBench subject consistency for the edited clip is within 10% of the unedited source clip, so we are not buying the score by destroying the video.

If those are not met we report it as a negative result together with the α dose–response curve and the selectivity map, which remain findings in their own right. A localisation that does not survive intervention is exactly the disagreement between "present" and "used" that section 5.2 says is worth looking for.

Baselines. A disentanglement score with nothing to compare against is an unanchored number. We run: **prompt-only** (same seed, new prompt, no patching — mandatory, and if we cannot beat it we have built an expensive way to do nothing); **MotionClone** as the training-free reference, chosen over the MOFT controller so the comparison is decided rather than optional, on the grounds that its public code is closest in spirit to our granularity 3; and **our own LoRA pair from 5.5** as the trained reference, which is cheaper and more directly comparable than running MotionDirector's full pipeline.

5.5 Deliverable 5: the trained model REWRITTEN

The course requires a trained or finetuned model. Our method is training-free, so the previous draft satisfied the requirement with a one-paragraph promise of "a small MotionDirector-style pair of LoRAs, one or two days in Stage 6". That was the vaguest item in the plan and the easiest to arrive at unprepared. It is now a numbered deliverable with a recipe, a dataset, a budget, a feasibility pilot and — the important change — **a job to do inside the project rather than beside it**.

What we train

Two LoRA adapters on the same AnimateDiff pipeline, fitted separately, following MotionDirector's dual-path design.

- **Appearance LoRA** — on the spatial path only: `to_q / to_k / to_v / to_out.0` inside the `transformer_blocks` (both `attn1` and `attn2`) of the frozen SD1.5 UNet. The key trick is *appearance debiasing*: train on a **single randomly sampled frame per step**, not the clip. A single frame contains no motion to learn, so the adapter is forced to absorb only identity, colour and texture.
- **Motion LoRA** — the same projection names but filtered to modules whose path contains `motion_modules`. Trained on the full clip with the appearance LoRA attached and **frozen**, so the appearance component is already explained away and the temporal loss carries residual motion. Latents are residualised against an anchor frame before the temporal loss, as MotionDirector does.

In `diffusers` plus `peft` this is a `LoraConfig` whose `target_modules` is a regex, and the spatial/temporal split is just whether `motion_modules` appears in the module path — the same string filter the E3 hooks already use.

SETTING	APPEARANCE LORA	MOTION LORA

Target modules	`transformer_blocks.*attn[12].to_{q,k,v,out.0}`	`motion_modules.*to_{q,k,v,out.0}`
Rank / alpha	16 / 16	32 / 32
Learning rate	1e-4, AdamW	5e-5, AdamW
Steps	300–500	500–800
Training input	one random frame per step	full clip, appearance LoRA frozen
Precision / batch	fp16, batch 1, gradient checkpointing on	
Clip geometry	8 frames × 256 × 256 for training, even though we generate at 16 × 512 — **to be confirmed by E5**, which is the single most likely thing to run out of memory	

What we train it on

Four motion concepts drawn from the verb vocabulary already frozen in section 4.1 — running, walking, jumping, spinning — with roughly five real clips each from UCF101 and Kinetics-400, so about 20 clips in total. These are real videos, which is a deliberate departure from the probing set and is disclosed in the note on page 4 and in the report's data section. Training on our own generated clips instead was considered and rejected: it would make the LoRA a distillation of the model's existing behaviour and the comparison below would be circular.

The part that makes it earn its place

A generic LoRA pair would satisfy the checklist and teach us nothing. Instead we train the motion LoRA twice and compare, which turns a required artefact into a test of Deliverable 2:

- **Full-coverage motion LoRA** — adapters on every temporal attention projection. This is the MotionDirector-equivalent and the trained baseline of section 5.4.
- **Probe-guided motion LoRA** — adapters only on the top-*k* coordinates from $S_{dyn}(L, t)$, which if H1 holds means `up_blocks[1]` and its neighbours. Same rank, same data, same steps, far fewer trainable parameters.

If the probe-guided variant matches the full-coverage one at a fraction of the parameters, our selectivity map has predictive power in *weight* space and not only in activation space, which is a considerably stronger claim than the patching result alone. If it does not match, that is a real negative result about the limits of the map, and a more interesting slide than "we also trained a LoRA". Either way it costs one extra training run, because the data, script and evaluation are already built.

A third view, nearly free. Take the trained full-coverage LoRAs and compute $\|\Delta W\|$ per module for the motion adapter against the appearance adapter. That gives a **training-derived layer ranking** to set against the two we already have: patching says what the model uses, the linear probe says what is present, and $\|\Delta W\|$ says where gradient descent chose to put the motion when asked. We report the rank correlation between all three. It is a dozen lines of code over weights we already have, and a three-way agreement (or disagreement) is the strongest single figure available to this project.

Feasibility and budget

E5 in Stage 1 records frames × resolution that fit, seconds per step and peak VRAM on a T4 — the training analogue of what E1 did for inference. The four training runs are then Stage 6 work at roughly 6 GPU-hours total; section 7 has the breakdown. We do not start LoRA training before the Stage 4 map exists, because the probe-guided variant depends on it.

5.6 Interface

A Gradio app. The important design decision is that it should expose the interpretability rather than hide it.

Inputs: two prompts (or two generated clips), a layer selector, a timestep range, and the patch strength α. **Outputs:** the edited clip, the three metric values, and the cross-attention heat map for the selected coordinate shown beside

the selectivity map from Deliverable 2, so a user can see why that coordinate was chosen. A demo that only shows the output video would satisfy the requirement, but it would waste the project.

Precomputed, not live — a practical decision the previous draft left open. At 94 s per clip, a live generation inside a demo is a 94-second silence in a graded presentation. The app therefore serves a cache of the ~500 clips and all precomputed maps, and every combination reachable from the controls is one we have already generated. Live generation stays available behind an explicit button with a stated wait, for the one or two cases where an examiner asks for something off the grid. Confirm the expected scope with the TA regardless; the brief explicitly says to ask.

5.7 Frozen protocol: the decisions register NEW

Everything below was implicit in the previous draft and is now fixed. The register exists so that a choice made in September is not quietly remade in October to suit the data, and so the stage-wise report can cite a decision rather than reconstruct one. Anything changed after this date is appended with a date and a reason rather than edited in place.

ID	QUESTION THAT WAS OPEN	DECISION
D-01	**What distance is d?**	1 − cosine similarity on the flattened activation. Content channel uses the static part $\bar{a}$, motion channel uses the frame-mean-removed part $\tilde{a}$. L2 reported as an appendix robustness check. §5.2
D-02	**One selectivity map or two?**	Two. S_{dyn} is the headline; S_{stat} is published beside it as the H3 control. §5.2
D-03	**How are pairs drawn?**	Anchor design. 104 within-family content pairs, 90 motion pairs, 56 cross-family content pairs held out as a labelled hard subset. §4.1
D-04	**How many seeds?**	Three — 42, 1337, 2024 — the full grid regenerated under each. Pairs never cross a seed. Across-seed spread is the error bar. §4.1
D-05	**E2 showed noun swaps drag the layout. What about it?**	Subjects split into two body-plan families; content pairs drawn within a family by default. Cross-family pairs reported separately, never pooled. §4.1
D-06	**What are the 15 subjects, 4 verbs, 2 scenes?**	Enumerated and frozen 22 September 2026, in the table on page 4. Also committed to `configs/probing_set.yaml`.
D-07	**Which timesteps?**	Summaries cached at all 25. Full tensors and all patching limited to $t \in \{0, 4, 8, 12, 16, 20, 24\}$, refined to ±2 around any peak. §5.1
D-08	**Which layers?**	Coarse pass: one representative module per block, 9 temporal and 7 spatial. Refined pass: all modules inside the winning block only. §5.1
D-09	**What exactly is stored?**	Channel means and variances, $\|\tilde{a}\|$ per channel, and attention maps head-averaged at 16 × 16 with the frame axis kept, for subject/verb/scene tokens. Full tensors only for flagged coordinates. §5.1
D-10	**"Most motion-selective channels" — by what score?**	$S_c(L, t, c)$, the same ratio restricted to one channel of $\tilde{a}$. Top k% with k swept over {1, 5, 10, 25} and reported as a curve. §5.3
D-11	**The null patch "must come back identical" — to what tolerance?**	Mean absolute latent difference < 1e-3 and decoded-clip PSNR > 45 dB. Bit-identity is unattainable in fp16 with cuDNN autotuning. §5.3
D-12	**Noising or resampling corruption?**	Resampling, from a real second clip. Stated explicitly because Zhang and Nanda show the two answer different questions. §5.3
D-13	**Is a patch one timestep or a range?**	A contiguous window, swept over {1 step, 5 steps, first third, all 25}. The winner from the coarse pass is what Deliverable 4 uses. §5.3

D-14	**D adds a CLIP cosine to a flow correlation. Are those commensurable?**	No. Both terms are min–max normalised across the evaluation set before entering D; raw values printed alongside. §5.4
D-15	**How many test pairs for D?**	Twelve, both directions, six per family, spanning all four verbs, listed in the report appendix. §5.4
D-16	**What result counts as success?**	Pre-registered three-part criterion in §5.4, including the both-directions requirement and the VBench guard. A failure is reported as a negative result, not reframed.
D-17	**MotionClone or MOFT?**	MotionClone. Decided, not "or". §5.4
D-18	**Does the demo generate live?**	No. Precomputed cache of ~500 clips; live generation behind an explicit button with a stated 94 s wait. §5.6
D-19	**What is the trained/finetuned model?**	A dual LoRA pair, plus a second probe-guided motion LoRA that tests the Deliverable 2 map in weight space. Full recipe in §5.5. Feasibility pilot E5 in Stage 1.
D-20	**Which checkpoint, exactly?**	Resolved base repository id and commit hash written into every result file, because `runwayml/stable-diffusion-v1-5` was withdrawn and the pipeline now falls through to a mirror. §4
D-21	**How is the linear probe validated?**	Split by scene, train on one and test on the other, then swap. PCA fitted on the training split only. Chance level plotted on every figure. §5.2
D-22	**How are H1 and H2 decided?**	argmax over blocks with a bootstrap CI for H1; a single changepoint fit with a bootstrap CI for H2. Both written into the analysis script before the sweep runs. §5.2

6. Week-by-week roadmap

Stage 0 and the midsem window are done. Everything from Stage 1 onwards is plan. Dates past 1 November depend on the assessment date being confirmed.

STAGE	DATES	FOCUS	MAPS TO
0	Sep 2–10	Scoping, reading groups 1–4, environment, pilots E1–E4.	—
—	Sep 11–20	Midsem window. Reading only, group 5, which changed the plan.	—
1	Sep 21–27	Prep presentation and recording. Freeze the protocol register (§5.7) and the probing vocabulary (§4.1). **Run E5, the LoRA feasibility pilot.** Pin checkpoint revisions.	Prep submission
2	Sep 28–Oct 4	Production extraction harness. Hooks on both paths, the coordinate triple, summary storage, the seed policy enforced in code. Generate the 360-clip probing set.	Deliverable 1
3	Oct 5–11	Coarse selectivity sweep: 16 representative coordinates × 25 timesteps × 3 seeds. Both S_{dyn} and S_{stat}. Linear probes.	Deliverable 2
4	Oct 12–18	Refined sweep inside the winning block. The layer × timestep map with bootstrap intervals. H1 and H2 decided by their pre-registered tests.	Deliverable 2
5	Oct 19–25	Patching framework. Granularities 1 and 2 first, then 3 and 4 if needed. All three controls, null patch before anything else. Sweep α and the patch window.	Deliverable 3
6	Oct 26–Nov 1	Edit demos in both directions across the twelve test pairs. All metrics against the prompt-only and MotionClone baselines. **Train all four LoRA runs; compute ‖ΔW‖ rankings.** Collect the qualitative grids.	Deliverables 4, 5
7	Nov 2 – assessment	Gradio interface over the precomputed cache. Stage-wise report. Package weights, code, README, environment file. Optional ModelScope robustness check if the schedule holds. Rehearse.	Final package

Where the slack is. Video inference and hook-based extraction are slower and more fragile than the equivalent image-diffusion work, and Stage 2 is the one most likely to overrun. If it does, take the time out of Stage 5, not Stage 3. The Stage 3 and 4 probing results are what everything else is built on; a rushed patching implementation is recoverable, a rushed sweep is not.

What moved in this revision. E5 moved forward into Stage 1, because discovering in late October that a LoRA will not fit in 16 GB would cost us Deliverable 5 entirely; the pilot costs half an hour now. The LoRA *training* runs stay in Stage 6 because the probe-guided variant cannot be built until the Stage 4 map exists. Stage 6 is consequently the densest week in the plan and is the second candidate, after Stage 2, for overrunning; if it does, the probe-guided LoRA is the part that survives and the full-coverage baseline is the part we cut, since the former is the one that answers a question.

Standing habit. The experiment log started in Stage 0 stays running to the end. Date, what was tried, model and layer configuration, result, next step. It is dull to maintain and it is the only reason the stage-wise report will be writable in Stage 7 instead of reconstructed from memory.

7. Compute budget NEW

The previous draft never totalled this, which meant the patching stage was quietly unaffordable: a naive sweep of 9 layers × 7 timesteps × 12 pairs × 2 directions is 1,512 generations, or 39 GPU-hours, on its own. The tiering in D-07, D-08 and D-13 exists because of this arithmetic, not for elegance. All figures use the 94 s per clip measured in E1.

ITEM	ARITHMETIC	GPU-HOURS	STAGE
Probing set generation with hooks	360 clips × 113 s (94 s + ~20% hook overhead)	11.3	2
Selectivity sweep	analysis over cached summaries, no generation	~1	3–4
Patching, coarse localisation	9 layers × 7 timesteps × 3 pairs × 1 direction = 189 runs	4.9	5
Patching, refined + α sweep	3 coordinates × 12 pairs × 2 directions × 5 α = 360 runs	9.4	5
Baselines	24 prompt-only clips + 24 MotionClone clips	2.1	6
LoRA pilot E5	100 steps, one concept	0.5	1
LoRA training	4 runs (appearance, motion full, motion probe-guided, spare) × ~500 steps	6.0	6
LoRA inference for evaluation	48 clips	1.3	6
Total	across Stages 1 to 6, six calendar weeks	**≈ 36.5**	

Against Kaggle's free quota of 30 GPU-hours per week this is comfortable, with the caveat that it is not evenly spread: Stage 2 and Stage 5 each want around 11 and 14 hours in a single week. The ModelScope robustness check is excluded and would add roughly 12 hours, which is why it stays conditional on Stage 7 having room.

8. Final deliverables checklist

Six items are due at the intensive assessment, separately from the prep presentation.

- **Trained or finetuned model.** The appearance LoRA, the full-coverage motion LoRA and the probe-guided motion LoRA from Stage 6, as `.safetensors`, together with the training script, the training configuration, loss curves, the probe-guided versus full-coverage comparison table, and the exact base checkpoint and commit hashes for the AnimateDiff motion adapter and the SD1.5 backbone actually resolved at run time. See §5.5.
- **Code with documentation.** Extraction harness, patching framework, metrics, LoRA training, interface. A README covering setup, dependencies (`torch`, `diffusers`, `peft`, checkpoint sources), the frozen `configs/probing_set.yaml`, and one command per figure so each result can be reproduced.
- **Interface.** The Gradio app described in §5.6, over a precomputed cache. Confirm the expected scope with the TA early, since this varies a lot by project.
- **Performance metrics.** Content fidelity, motion fidelity, the disentanglement score D, and the prompt-only, MotionClone and LoRA baselines. Both edit directions reported, and the §5.4 success criterion evaluated explicitly rather than left to the reader.
- **Qualitative results.** Side-by-side grids: original A, original B, content of A with motion of B, and the reverse. Non-negotiable given the project is visual.
- **Stage-wise report.** Why each step was taken, the major experiments and what they showed (including the ones that failed), how the probing set was built, the decisions register from §5.7, and the interface methodology. More detailed instructions are promised by the course; the experiment log is what makes this writable.

9. Prep presentation and video checklist

Slides, against the course instruction sheet:

- 25 slides maximum. The companion deck is at 25 exactly, so any addition needs a deletion. The 22 September revision added a trained-model slide and a frozen-protocol slide, paid for by merging the status slide into the

roadmap slide and compressing the references from three slides to two.

- Title slide with the topic, an explicit statement that this is IE 643 course project work, the team name and every member's roll number. **Roll numbers still need filling in**, on the title slide and the closing slide.
- Outline slide.
- Background materials discussed rather than listed.
- Approaches tried so far and the future plan, broken into subtasks.
- Diagrams for the hard concepts: the U-Net factorisation, the patching mechanism, the selectivity map.
- Data collection and curation described concretely — now including the frozen vocabulary and the pair counts.
- Models investigated and how each will be used.
- Pretraining data of the models used (§4.1), plus our own probing set and why it is built the way it is.
- Experiments carried out so far. E1 to E5 are on the deck; E3's per-clip storage figure and E5's outcome still need filling in from our own logs.
- References in the exact required format.

Video:

- 15 minutes maximum. There is a penalty for going over, so rehearse against a timer rather than a feeling.
- Every team member presents and is on camera throughout, not only during their own section.
- Do not read the slides out. The instruction sheet says this twice and it is the easiest mark to lose.
- OBS Studio for recording (https://obsproject.com/wiki/install-instructions). Scene with slide capture plus a webcam overlay. Do one full test take before the real one and check audio levels and lighting on it.

10. Submission

- Google Drive folder named exactly `Claudnt_IE643_CourseProject_Prep`, created with IITB email and SSO. The team name has an apostrophe in it, which Drive and the course form may not handle cleanly, so drop it in the folder name and keep the real spelling on the slides.
- Both the slides and the video go in that folder.
- Share it so the instructor and every TA can open it. Test in an incognito window rather than assuming. Inaccessible content is not considered submitted.
- Submit the folder link through the course form once it is posted.
- Confirm the revised prep presentation deadline separately from the assessment date. The two were not necessarily moved together.

11. Open questions for the TA

Shorter than in the previous draft. Four of the six earlier questions were things we could decide ourselves and have now decided; what remains is genuinely outside our control, plus one proposal we would like confirmed rather than assumed.

QUESTION	WHY IT MATTERS TO US, AND WHAT WE HAVE ASSUMED MEANWHILE
Firm dates for the prep presentation and the intensive assessment.	The note only said "about a month and a half". Everything past 1 November in §6 is provisional. We have assumed early-to-mid November and are treating the 21 September prep deadline as live.
Is GPU time provided through the course, or do we source our own?	This sets the resolution of the Stage 3 sweep more than any other single factor. We have budgeted against Kaggle's free 30 GPU-hours per week and §7 shows the plan fits in ≈ 36.5 hours total. More compute would buy the ModelScope robustness check and a finer timestep grid.

Is one primary model enough for full credit, or is cross-model validation expected?	Changes how much of Stage 7 we reserve. Assumed: one model is enough, with ModelScope as a bonus. If cross-model validation is expected we need to know by mid-October, not in November.
What counts as an acceptable interface — a live Gradio app, or an annotated notebook?	The brief tells us to ask. Assumed: Gradio over a precomputed cache (D-18), which we think is the honest form given 94 s per clip.
Does the LoRA work in §5.5 satisfy the trained/finetuned model deliverable?	This is a proposal we would like confirmed rather than an open design question — §5.5 is now a full specification and we are proceeding on it. We believe it qualifies: real trained weights, on a named base checkpoint, with a training script and curves. If a fuller finetune is expected instead, that changes the compute plan in §7 materially and we would need to know before Stage 4.
Any required structure or rubric for the stage-wise report beyond what has been announced?	We are writing against the six-item checklist in §8 and keeping the experiment log in a form that maps onto it.

12. What changed on 22 September, in one page

For anyone reading this against the 21 September draft. The substance of the project is unchanged; what changed is that the choices it depended on are now written down.

SECTION	CHANGE
§2.2 Pilots	E1's measured numbers filled in (94 s, 10.2 GB). E3's storage figure filled in (3.4 GB per clip). **E5 added** — LoRA feasibility, Stage 1.
§4.1 Probing set	Vocabulary enumerated and frozen. Pair-construction rule made deterministic with counts. Seed policy fixed at three seeds. Body-plan families added in response to E2. Pretraining-data paragraph added. Real-video use for §5.5 disclosed.
§5.1 Harness	Two-tier sweep made explicit. Storage contents specified exactly. Checkpoint-revision recording added.
§5.2 Selectivity	**Distance function defined.** One map became two (S_{dyn}, S_{stat}), which is what H3 actually implies. Per-pair aggregation and bootstrap intervals specified. Pre-registered tests for H1 and H2 added. Linear-probe protocol specified with its split and chance level.
§5.3 Patching	Per-channel score defined. Corruption type stated. Patch window added as a swept parameter. Null-patch tolerance made numerical.
§5.4 Evaluation	Commensurability of D corrected by normalising both terms. Test-set size fixed at twelve pairs. **Success criterion pre-registered.** Training-free baseline decided (MotionClone).
§5.5 Trained model	**Rewritten from one paragraph to a full deliverable.** Recipe, targets, hyperparameters, data, and the probe-guided variant that makes it test Deliverable 2. $\|\|\Delta W\|\|$ as a third view.
§5.6 Interface	Precomputed-versus-live resolved.
§5.7 Register	New. Twenty-two decisions, each pointing at the section that implements it.
§6 Roadmap	E5 moved into Stage 1. Stage 6 rebalanced and its cut-order stated.
§7 Budget	New. Totals ≈ 36.5 GPU-hours and shows why the sweep had to be tiered.
§11 Open questions	Trimmed from six to what is genuinely outside our control; the LoRA question reframed as a proposal to confirm.

Appendix A. The still-blank list

Every remaining placeholder in the project, in one place, so none of them is discovered the night before the recording. Nothing here is estimated; each gets filled from our own logs or from a decision that is genuinely someone else's.

WHAT IS MISSING	WHERE IT APPEARS	HOW IT GETS FILLED	BY WHEN
Roll numbers	Deck slides 1 and 25; §1 of this document	From the team	Before the recording
E5 outcome	Deck slide 10; §2.2	Run the pilot on Kaggle	Stage 1, by 27 Sep
Confirmed assessment date	§1, §6	TA	Outside our control
Checkpoint commit hashes	§4, D-20	Recorded automatically by the generation script once written	Stage 2
Real frames in the Deliverable 4 grid	Deck slide 17	Stage 6 output replaces the four placeholders	Stage 6
Measured selectivity map	Deck slide 15 (currently drawn by hand)	Stage 4 output replaces the illustration	Stage 4

Appendix B. Risks, and what we do about them

Compute limits the layer × timestep sweep

Start coarse — one module per block, seven timesteps — and refine only around what looks promising. §7 shows the tiered plan fits in ≈ 36.5 GPU-hours where the naive version would have needed over 60.

AnimateDiff's split is too easy to be a finding

Fair criticism, and we say so up front. The answer is ModelScope T2V, where spatial and temporal were trained together. If the same structure shows up there, it generalises. The probe-guided LoRA in §5.5 is a second, cheaper answer to the same objection: it tests whether the map predicts anything, rather than whether the map merely reflects the architecture.

Everything looks content-selective and we find nothing

This is H3, and it is the failure mode we most expect. Subtract the per-frame mean before scoring, and publish S_{stat} beside S_{dyn} so the confound is demonstrated rather than asserted. Reading Xiao et al. saved us a week here.

Patched outputs come out incoherent rather than wrong

Run the null patch and the random-coordinate control before believing anything. Prefer scaled injection over hard replacement, and report the whole α curve. The VBench guard in the §5.4 success criterion exists so that an incoherent clip cannot score well.

The LoRA does not fit in 16 GB

E5 in Stage 1 is precisely this check, moved forward from Stage 6 for that reason. Fallback ladder: 8 frames at 256 × 256, then rank 8, then LoRA on the motion path only with the appearance path handled by prompt conditioning instead.

Stage 6 overruns

It is the densest week in the plan. Cut order is stated in §6: the full-coverage LoRA baseline goes first, the probe-guided LoRA is kept, because only the latter answers a question.

A decision gets quietly remade to suit the data

The §5.7 register is dated and append-only. Any change after 22 September carries a date and a reason.

> This document supersedes the draft of 21 September 2026. The companion deck, `IE643_Prep_Presentation.pptx`, was revised on the same date and is consistent with it.